ChatGPT

Robust Wear-Leveling under Adversarial Flash Workloads

Introduction and Challenges

Flash-based SSDs require **wear leveling (WL)** to distribute erase cycles uniformly and maximize lifespan [1]. Without robust WL, adversarial access patterns or malicious workloads can **prematurely wear out** some blocks while others remain underutilized [2]. We consider four challenging patterns: (1) a **single hot LBA** (or few hot addresses) receiving disproportionate writes, (2) a **Zipfian skew** where a long-tail distribution causes a small fraction of LBAs to get most writes, (3) **FTL probing** that cycles through many LBAs to exhaust mappings (forcing the FTL to allocate new blocks continuously), and (4) **mixed multi-tenant workloads** with high-frequency (heavy writer) and low-frequency (light writer) tenants. The goal is to design a WL algorithm that **minimizes wear imbalance** (low *min–max erase count* gap and low *erase count variance*), maintains high **wear fairness** (e.g. Jain's index close to 1), and avoids degrading performance (**low per-flow slowdown**, high throughput, and low tail latency). Crucially, the solution must **preserve tenant fairness** and **low latency** even while leveling wear.

Adversarial Wear Patterns: Each pattern stresses WL differently:
- *Hot-spot writes:* A single LBA (or small set) being continuously written can concentrate erases on a few physical blocks (targeted wear-out).
- *Zipfian skew:* Access follows a heavy-tailed distribution (few blocks "hot", many "cold"), risking uneven wear if hot data stays on the same blocks.
- *FTL probing:* Sequential or cycling writes across the entire LBA space attempt to **use every block** and defeat naive mapping strategies, potentially causing excessive garbage collection and wear amplification.
- *Mixed workloads:* One tenant's intense writes can induce frequent GC and wear on their blocks, while other tenants do little writing (or mostly reads). This can cause **wear imbalance across tenants** and interference (the heavy writer's background GC might impact the light tenants' latency).

Wear Metrics: We will evaluate WL using: (a) the **difference between max and min erase counts** (should be minimal), (b) the **variance of erase counts** across blocks (lower is better), and (c) **Jain's fairness index** for wear distribution (close to 1.0 if uniform). For multi-tenant scenarios we also track **per-flow slowdown** (each tenant's performance degradation compared to running alone) and standard performance metrics like **throughput** and **tail latency** (99th percentile), especially if garbage collection (GC) is modeled.

(Table 1 will summarize these metrics and adversarial patterns – placeholder).

Wear-Leveling Algorithm Design Overview

Goals: The WL algorithm must **even out block erasures** over time while **minimizing overhead**. It should proactively prevent any block from being over-used (addressing hot-spot and Zipf skews) and reactively correct emerging imbalance (addressing long-term drift). At the same time, it should **respect tenant**

isolation: one tenant's heavy writes should not unfairly degrade another's performance. The design balances **aggressiveness** (uniform wear for lifetime) with **restraint** (avoid constant data migration that adds latency).

Key Strategies:
- **Dynamic Allocation Bias:** Always direct new write operations to the **least-worn available block** (or one of the least-worn) to naturally equalize wear. This greedy leveling ensures the most worn block is avoided until others catch up [1]. It helps counter both hot-spot and Zipf skew patterns by shunting incoming writes to cooler blocks rather than repeatedly using the same physical block.
- **Hot/Cold Data Separation:** Logically **classify blocks as "hot" or "cold"** based on how frequently their data is updated [1]. Hot blocks (frequently written) and cold blocks (holding mostly static data) are tracked separately (a *dual-pool* approach). This allows targeted interventions: when a hot block's erase count grows too high relative to a cold block, we can **swap their contents** [3]. By moving the hottest data into the least worn (coldest) block, we *spread out* future writes and increment the cold block's wear, keeping the wear gap bounded. This addresses extreme cases like a single hot LBA: its data will be rotated through many physical blocks over time instead of destroying one block.
- **Wear Leveling Trigger:** Define a **wear imbalance threshold** (e.g. max erase count minus min erase count > Δ, or variance above a set value). Periodically (e.g. every certain number of writes or at fixed time intervals), check wear distribution. If imbalance exceeds the threshold, initiate a **wear-leveling cycle**: find the most worn block and the least worn block, and perform a *hot-cold swap* of their data (or otherwise redistribute load) [4]. This periodic check prevents constantly moving data (which would hurt performance) and instead performs leveling in batches. The threshold and interval are tuning knobs – too frequent triggers cause excessive overhead, too infrequent allows large imbalance [4]. Our design chooses a moderate interval and Δ to ensure near-uniform wear without oscillation.
- **Randomization & FTL Resilience:** Introduce some **randomness in block selection** for writes (among the lower-wear blocks) to prevent an adversary from predicting or exploiting the allocation pattern. For example, rather than *always* picking the absolute least-worn block, pick a block from the bottom X% of wear counts at random. This defends against a clever "FTL probing" adversary who might otherwise craft writes to manipulate which block gets used next. Randomization also avoids synchronized behavior where multiple hot LBAs might all target the single least-worn block in lockstep.
- **Fairness and Isolation:** To preserve tenant QoS, the algorithm performs wear-leveling **gradually and smartly** so that one tenant's heavy load does not unduly pause another's I/O. For instance, WL swaps or data migrations can be done in the **background** or during idle periods if possible. If a heavy-write tenant is causing frequent leveling actions, those actions can be throttled or staggered to limit impact on a light tenant's requests. In a multi-channel SSD, we may also exploit **spatial isolation** – e.g. confine each tenant primarily to certain channels or block groups for short intervals and then rotate assignments across channels over time. This way, at any moment tenants don't interfere, but over long periods each tenant has used all channels evenly (equalizing wear across channels/dies) [5]. The algorithm thus **allows a small, controlled amount of wear imbalance** temporarily to preserve performance isolation, and corrects it slowly over time [6]. This idea is inspired by FlashBlox, which permits limited wear differences to gain performance isolation, then evens out wear at coarse intervals [5].

Pseudocode of the Wear-Leveling Algorithm

Below is a high-level pseudocode outlining how the algorithm operates within the FTL. This assumes a page-level mapping and tracks per-block wear counts and validity. (It can be adjusted for a simplified model without full GC as noted later.)

```
Initialize wear_count[block] = 0 for all physical blocks
Initialize free_block_list = all blocks
Initialize mapping[LBA] = None (logical to physical mapping table)
Set WL_check_interval = N (e.g., check every N write operations)
Set wear_threshold = Δ (max allowed difference in erase counts)

on WriteRequest(LBA, size):
    num_pages = ceil(size / PAGE_SIZE)
    for i in 1 to num_pages:
        if mapping[LBA] != None:
            old_block, old_page = mapping[LBA]
            mark old_block.page[old_page] as invalid  // out-of-place update
        # Select target block for new write:
        target_block = select_block_for_write()
        target_page = target_block.next_free_page
        program(target_block, target_page, data)
        mapping[LBA] = (target_block, target_page)
        if target_block is now full:
            // If no GC, we could immediately erase, or defer until needed
            enqueue full block for garbage_collection

    increment global_write_count
    if global_write_count % WL_check_interval == 0:
        perform_wear_leveling_cycle()

function select_block_for_write():
    # Prefer least-worn blocks that have free space
    candidate_set = {b in free_block_list or not-full block | wear_count[b] is
among lowest X%}
    # choose a random block from candidate_set
    return block_choice

function perform_wear_leveling_cycle():
    max_block = block with highest wear_count
    min_block = block with lowest wear_count
    if wear_count[max_block] - wear_count[min_block] > wear_threshold:
        // Swap static data (cold) from min_block into max_block, free min_block
for new writes
        data_to_move = read_all_valid_data(min_block)
        for each page in data_to_move:
            write that page into max_block (or another suitable block)
        erase(min_block);  wear_count[min_block] += 1
        // After swap, min_block (was cold & least worn) takes over writes from
max_block
        // max_block now holds cold data and can be left alone to "cool down"
        update free_block_list etc.
```

Explanation: Every write is directed to an appropriate physical block via `select_block_for_write()`. This function implements the **greedy + random** selection: it filters blocks with the lowest wear (and with free pages) and randomly picks one, ensuring we don't always hit the exact same block. If the target LBA had existing data, the old page is invalidated (simulating out-of-place writes as in real SSDs). We keep a count of writes and every `WL_check_interval` operations we call `perform_wear_leveling_cycle()`. In that cycle, if the *wear gap* (max minus min erase count) exceeds our threshold, we execute a **hot–cold swap**: take the most worn block (max_block, presumably containing hot data) and the least worn block (min_block, likely holding cold/static data). We migrate all valid data off the least-worn block (so it becomes free) – effectively moving any cold data it had into a more worn block – then **erase** the least-worn block (giving it one more erase so it "catches up" a bit) and put it back into service for new writes. The hot data that was on the max_block can now start going into this freshly erased block. This process ensures the block with the smallest wear count gets used and its count increases, while the previously hottest block is relieved from further erases for a while (it now holds cold data after swap). Over time, this **maintains wear counts within a tight range** [4] [3].

Handling Specific Adversarial Patterns:

- **Hot LBA (targeted wear):** The algorithm detects rapid reuse of the same LBA. Each update to that LBA goes to a *new physical page*, and by selecting different blocks (due to the least-wear policy), it **rotates the LBA across many blocks** rather than one. If one block starts accumulating much higher wear from this LBA, the periodic check will trigger a swap to move the LBA's data to a cooler block. Thus no single block services all writes for that hot LBA; the wear is spread among a pool of blocks. For instance, if an attacker writes one address in a tight loop, the FTL might cycle it through a dozen different physical blocks in round-robin fashion to even out erases.
- **Zipfian Skew (long-tail):** By always favoring less-worn blocks, **hot addresses naturally get placed into blocks that were less used so far**, balancing things. The multiple hot addresses in the heavy tail will each be allocated into different blocks over time (since one block's wear will quickly rise, causing subsequent writes to divert to other cooler blocks). Additionally, if some blocks remain largely cold (holding rarely-updated data), occasional swaps will migrate those cold data blocks into the hot pools so that even "cold" blocks eventually get erased and no block is left with zero wear. This prevents extremely skewed scenarios where hot blocks have 1000+ erases while some cold blocks sit at 1–2 erases. The **variance and min–max gap** stay low because cold blocks are periodically sacrificed (erased) to level the field.
- **FTL Probing (cycle through LBAs):** An attacker writing sequentially through the address space will force the FTL to allocate new blocks continuously. Our strategy copes by **randomizing block allocation** and maintaining a reserve of free blocks. Even if the attacker touches every LBA (using every block), wear will be evenly distributed by design (since each new block used was the least-worn available). The algorithm also ensures that once the device is near full (all blocks have some valid pages), it will start **garbage collecting** and reusing blocks in a wear-aware manner. We would configure the garbage collector to always recycle one of the more worn blocks first (if it has invalid pages) to avoid stranding wear on one block. Additionally, by monitoring free block count, we can trigger GC *early* (before fully exhausting all blocks) to avoid a burst of erases all at once. The net effect is that a probing workload yields a uniform wear pattern and avoids pathological GC stalls.
- **Mixed Workloads (High vs Low Intensity Tenants):** The WL algorithm, combined with an appropriate scheduler, will isolate tenants in the short term but **balance wear in the long term**. For example, suppose Tenant A writes at a high rate and Tenant B at a low rate. In the short run, many of the erases will occur in blocks serving Tenant A's data. Our algorithm can allow this *temporarily* (for performance isolation) but will gradually integrate Tenant B's blocks into rotation. If Tenant B has blocks with very low wear, during a wear-level cycle the algorithm might move some of Tenant B's cold data into a high-wear block used by Tenant A,

and free up B's block for use by A's writes. This way, B's block gets an erase (bringing its wear closer to A's blocks), and A's hottest block is relieved. To prevent undue impact on Tenant B, such swaps could be done infrequently or during idle times. Overall, each tenant eventually contributes to wear proportional to their usage, but **no tenant's data remains on exclusively low-wear blocks forever**. Performance fairness is preserved by not constantly interfering with Tenant B's I/O: the background leveling is tuned to have *negligible impact on foreground latency* [5]. In practice, this means scheduling wear-leveling operations when channels are free or using partial block swaps so as to amortize the overhead. The result is that even if Tenant A writes all data to a single channel initially and Tenant B only reads on others, the mechanism can achieve ~95% of ideal lifetime with minimal disruption [5] – i.e., near-uniform wear across channels/dies by eventually migrating Tenant A's workload around, while Tenant B sees little performance loss.

(Figure 1 will illustrate the wear distribution (erase counts per block) for a hot-spot workload with and without the new algorithm – placeholder).
(Figure 2 will illustrate multi-tenant wear balancing: e.g., wear over time on each tenant's blocks, showing Tenant A's wear leveling off across all blocks – placeholder).

Implementation in Simulation Environments

1. Integration with the SSD Fairness Simulator

The existing fairness simulator uses a simple SSD model (no detailed FTL or GC by default). We will extend it by implementing a basic FTL layer with wear leveling. According to the project guidelines, this means modifying the `SSD` class in `ssd.hpp` / `ssd.cpp` to incorporate the new behavior [7]. Key steps for implementation:

- **Data Structures:** Add fields to the `SSD` model for physical geometry and wear tracking. For example, define constants for **number of blocks** and **pages per block** (these can be configurable via `SimConfig`). Add an array `wear_count[block_index]` initialized to 0, and perhaps a 2D array or vector to represent pages in each block (to track used/free pages and maybe valid/invalid count). Also maintain a **free block list or free page count** for allocation. If memory is a concern, we can abstract and not store every page, but at least track how many pages of each block have been written and how many are invalid.

- **Mapping & Allocation:** Implement a simple **logical-to-physical address mapping**. Since the simulator's trace includes an `address` field (which was previously unused), we can use the lower bits of the address (or an LBA number derived from it) as the logical page index. For simplicity, treat each write in the trace as writing one page (or a fixed number of pages corresponding to the request size). In `SSD::dispatch` or a similar function intercepting writes, do the following:

- Determine the logical page(s) being written from the request's address and size.
- For each page, check if it was written before (we can keep a map `LBA -> (block, page)` for valid mapping). If so, mark that physical location as invalid (simulating that the old data will be eventually garbage-collected).
- Select a physical target for the new write using our **wear-leveling allocation** policy (e.g., find the block with the minimum `wear_count` that has free space, using a small random offset as

described). Write the data to the next free page in that block and update the mapping. If this fill causes the block to become full, mark it as needing erase.

- Optionally, if GC is not modeled, we might **immediately erase** a block when it's full to free it (incrementing its wear_count). This is a simplification that ensures we continue to have free blocks. Alternatively, implement a basic GC: when free blocks drop below some threshold, pick a block with many invalid pages, migrate any valid ones elsewhere, erase it, and update wear_count. The wear-leveling policy can guide GC's choice: e.g., prefer erasing a *relatively worn* block first to give it a "rest" if possible, or simply choose the block with most invalid pages (greedy GC).

- After each write (or each few writes), update a counter and periodically trigger the **wear-leveling check**. If the wear gap is too high, perform the block swap procedure: identify the block with max wear and min wear. In code, this could mean moving any remaining valid pages from the min-wear block into other blocks (one of which might be the max-wear block) and then erasing the min-wear block. We'd increment its wear_count and put it back as a free block. This logic might be implemented in a helper method within `SSD`.

- **Maintaining Performance Timing:** We must ensure these new steps fit into the simulator's event model. The simulator currently computes service times for each request (based on fixed read/write bandwidth and number of channels) and uses an event queue. We should account for any additional latency due to wear-leveling operations. For simplicity, we can assume that the wear-leveling (swaps, extra writes for moving data) imposes a fixed overhead per operation or per swap. Since the fairness simulator doesn't model micro-level timing of flash operations, one approach is to count each additional internal operation (like a page migration or block erase) and translate it into an approximate time penalty. For example, if an erase takes E microseconds and a page copy takes C microseconds, a wear-leveling swap that moves k pages would cost ~k*C* + *E time in total. We could deduct some performance or log it for stats, but to avoid complicating the simulator, we might simply record these overheads in metrics (e.g., count extra writes and erases as part of write amplification stats) without actually delaying the event completion times too much (since that could interfere with the scheduling fairness logic). A balanced approach is to simulate minimal latency impact***** in the fairness sim (consistent with "negligible disruption" assumption [5]), and let MQSim capture the real latency effects in detail.

- **Metrics Collection:** Extend the simulator's metrics to output wear-related data. We will add fields such as *max_erase_count*, *min_erase_count*, *erase_count_variance*, perhaps computed at end of simulation or at periodic checkpoints. We should also log *per-user slowdown* (which can be derived by comparing each user's achieved throughput or average latency in the shared run vs a baseline). Jain's fairness index can be computed for throughput or slowdown across tenants and output as well [8]. This might involve updating the `Metrics::UserStats` and the CSV output to include these new values.

Overall, the fairness simulator integration is about **50-100 lines of C++ changes**: defining new arrays and logic in `SSD::dispatch` and perhaps a new method for the WL cycle. We will carefully test it on small traces to verify that wear counts behave as expected (no block gets way ahead of others, etc.). *Note:* Because this simulator abstracted away physical details, our implementation is an approximation. It may not capture all nuances of a real SSD, but it will allow us to experiment with wear-leveling policies in a multi-tenant setting and observe the effects on fairness metrics.

2. Integration with MQSim (Realistic SSD Model)

In MQSim (a detailed SSD simulator), many of these features – mapping, GC, wear tracking – are already present. MQSim models multiple channels, chips, dies, and a full FTL with garbage collection. To implement our custom wear-leveling in MQSim, we will take advantage of its extensibility:

- **FTL Policy Customization:** We will modify or extend MQSim's FTL module to incorporate our **wear-aware allocation**. In MQSim's code, there is typically a function that selects a free page (or block) for a write (often part of a page allocation or write frontier management). We will inject logic to bias that selection toward blocks with lower erase counts. MQSim may not directly expose erase counts to the mapping policy, so we might maintain an array of block erase counts (which MQSim updates on each erase) and use it in the allocation decision. This might involve subclassing or editing the `Address_Mapping_Unit` or `BlockManager` components to sort or choose free blocks by wear.

- **Channel/Dies Wear-Leveling:** MQSim, by default, might allocate across channels evenly for performance. Our algorithm should ensure **wear is balanced not just within each chip but across channels and dies**. If MQSim isolates workloads to separate channels (for performance isolation), we will implement a mechanism to periodically **rotate a portion of workload from one channel to another** (or internally migrate some data) to equalize wear. For example, if one channel has serviced far more writes (higher average erase count) than another (perhaps because one tenant bound to it was very write-heavy), we could schedule a background task in MQSim that moves some logical units (e.g., logical block ranges or whole flash die assignments) between channels. In practice, implementing this exactly might be complex in MQSim, but we can approximate it by occasionally shuffling which channel new writes go to for a given logical range, or by artificially injecting cross-channel copy operations (using MQSim's support for copy-back operations if available). The **analytical model in FlashBlox** can guide the target imbalance: we allow slight differences short-term, then gradually converge [6].

- **Garbage Collection (GC):** MQSim's GC policy can be tuned to complement our WL. We can set a GC trigger threshold (e.g., start GC when free block count < 5% of total) so that adversarial patterns can't completely exhaust free blocks. Moreover, we can modify GC's victim block selection. Instead of purely picking the block with most invalid pages, we might pick among the top N invalid blocks the one with the **highest erase count** (to let it rest) or the one with lowest count (to force it to catch up, depending on strategy). Our WL algorithm likely prefers to **garbage-collect colder, less-worn blocks first** (like static wear leveling): this forces those blocks to be erased and leveled. However, constantly GC'ing low-wear blocks might increase write amplification. A balanced approach is: use MQSim's existing GC (which tends to reclaim hottest blocks that are full of invalid pages) and rely on our separate wear-leveling routine for static data if needed. We will instrument MQSim to periodically check wear distribution (maybe on each GC invocation or every T seconds of simulated time) and if imbalance > threshold, perform a controlled migration akin to our swap: choose a low-wear block with valid data and a high-wear block, and migrate data between them. This can be done by issuing internal copy commands or marking blocks for relocation. MQSim allows scheduling such internal operations if we emulate them as background I/O or maintenance requests.

- **Metrics in MQSim:** We will configure MQSim to output detailed stats on wear. MQSim typically reports per-block erase counts, overall wear distribution, and performance metrics per I/O queue. If not already available, we'll instrument the code to dump erase count histogram at end of simulation.

The same metrics will be gathered: min/max erase, variance, maybe Gini or fairness index of wear, plus per-flow performance (throughput, latency) which MQSim can provide per I/O queue or trace. Jain's fairness index for performance can be calculated in post-processing from throughput numbers. Tail latency (99th percentile) for each flow will be measured to see the worst-case delays (especially to observe if heavy GC for one tenant affected another's read latency).

Validation in MQSim: We will test that with our modifications, MQSim indeed shows improved wear balance under adversarial workloads. For instance, if we simulate the one-hot-LBA attacker, a stock FTL might concentrate erases on a few blocks (until they maybe fail if endurance was modeled), whereas our modified FTL will distribute erases widely, showing a much lower min-max difference. We also expect to see in MQSim that performance isolation is maintained or improved: e.g., in a mixed workload, the heavy writer's GC doesn't cause as many read tail latency spikes for the light reader because we either partition workloads or perform leveling slowly. **Flash endurance** is not explicitly enforced here (we won't simulate block failures given no endurance limit), but we'll extrapolate that a more uniform wear means **longer time until any block hits its limit**.

(Figure 3 to include a diagram of the algorithm architecture in the simulator vs MQSim integration – placeholder.)
(Table 2 to compare key implementation differences between the two simulators – placeholder.)

Experimental Evaluation Plan

Workloads and Trace Generation

We design a comprehensive set of workloads corresponding to the adversarial patterns, plus combinations, to evaluate the wear-leveling algorithm. Each workload will be represented as an **I/O trace** (a CSV of I/O requests) which we can feed to both the fairness simulator and MQSim (with appropriate format conversion). We outline the workloads and how to generate them:

1. **Hot-Spot Write Pattern:** One LBA (or a very small set of LBAs) is written intensely. For example, a trace with a single logical address (say LBA 0) receiving a write request every 10 µs, for a large number of operations (to cause dozens of erases on that block). We can generate this by fixing the address field for most requests. Optionally, include a few other addresses with minimal writes to act as background. *Trace generation:* e.g., use a Python script to emit entries like `(timestamp, process_id, user_id, WRITE, address, size)` where `address` is constant (`0x0000` for LBA0) for 90% of the writes, and a few other addresses for the remaining 10%. Ensure the total number of writes is enough to exceed one block's capacity multiple times (to force multiple erasures). This pattern specifically tests if the algorithm moves that hot LBA across blocks.

2. **Zipfian Distribution Writes:** A large set of LBAs (hundreds or thousands) where write frequency follows a Zipf (power-law) distribution. For example, if 1000 distinct addresses are used, the top 10 addresses might account for 40% of writes, the next 40 addresses 30%, and so on. We will generate addresses using a Zipf generator (with a chosen skew parameter like 1.2 for a moderately long tail). *Trace generation:* we can assign ranks to addresses 0–999 and sample from a Zipf(α) distribution to pick an address for each request. All requests are writes (to maximize wear). This trace will reveal how the WL algorithm handles many hot blocks versus many cold ones.

3. **FTL Probing Sequential Writes:** A workload that sequentially (or cyclically) writes through a large logical address range to try and use every physical block. For instance, writing to LBA 0, 1, 2, ..., N in order, then repeat from 0, multiple times. This ensures every block gets some writes, and the attacker attempts to *exhaust the free blocks*. *Trace generation:* we pick an address range (say 0 to N-1, where N corresponds to the total number of pages or logical blocks in the device) and generate write operations in increasing order. After reaching the end, loop back to the beginning if more operations are needed. We might include a short think time or use multiple threads to simulate parallel access if needed. The size of N should be such that the device would be almost filled by one pass (to stress GC). For example, if the device has 100 blocks * 256 pages each = 25600 pages, we might use N=20000 unique logical pages to ensure nearly full usage. This trace will test the algorithm's ability to keep wear balanced when *every* block is actively used, and check that it avoids a situation where all blocks fill up at once causing a massive GC.

4. **Mixed Tenant Workload:** Combine a high-intensity write workload with a low-intensity or read-heavy workload. For example, two tenants: Tenant A generates 10K write IOPS to a range of addresses (could be sequential or random within its range), and Tenant B generates 1K read IOPS to a different range (or writes very infrequently). We'll map each tenant's I/Os to distinct user_ids in the trace to distinguish them. *Trace generation:* we can produce two streams and intermix them by timestamp. Tenant A's stream: e.g., writes 4KB every 100 µs to random addresses in 0–N (or maybe with some hot spots). Tenant B's stream: reads 4KB every 1000 µs (10× less frequent) to random addresses in another range (or could even read from the same range A writes to if we want to simulate interference on shared data – but that complicates wear which only writes cause). Alternatively, we can have Tenant B also do writes but at a far lower rate. The key is to have a disparity in volume: perhaps Tenant A accounts for ~90% of writes, Tenant B 10%. We can also introduce a third or fourth tenant for more complexity (e.g., one purely reading, one doing small writes, one doing large sequential writes, etc.). This mixed trace will let us measure **tenant fairness** (slowdowns and latency) and see if the wear-leveling algorithm unfairly impacts one tenant. For example, does Tenant B (light user) see latency spikes due to Tenant A's GC or wear leveling? Does the algorithm still level wear even though one tenant hardly writes (meaning it might have to occasionally force an erase on Tenant B's blocks to keep up)?

For each of these synthetic traces, we'll vary parameters like **request size** (4KB vs larger writes), **access locality** (sometimes use random addresses vs sequential within the distribution), and **duration** (enough operations to observe steady-state wear). The simulator's `trace_gen.py` can be adapted or we can directly script these patterns. We will ensure traces are long enough for meaningful results – e.g., enough total writes to cause at least, say, 50 block erasures in the baseline, so differences are measurable.

(Table 3 will list workload parameters: number of tenants, R/W mix, hot fraction, trace length, etc. – placeholder.)

Experimental Procedure and Metrics Collection

We will run each workload in two modes: **Baseline** (wear leveling OFF or using a naive/default policy) and **New Algorithm** (wear leveling ON). In the fairness simulator, "WL OFF" essentially means the FTL uses a trivial allocation (e.g., always write to the same block until full, then next block, with no swapping). In

MQSim, baseline means using its standard wear leveling (which might be limited to dynamic leveling). By comparing these, we can quantify improvements. Key metrics and how we'll measure them:

- **Wear Imbalance:** At the end of the simulation (or at periodic checkpoints), we gather each physical block's erase count. We will compute *min*, *max*, *range (max–min)*, *mean*, *variance*, and *standard deviation*. We'll also compute **Jain's fairness index** for the set of erase counts as $J = \frac{(\sum_i E_i)^2}{N \sum_i E_i^2}$, where E_i is the erase count of block *i* and *N* is total blocks. (An index closer to 1 indicates very uniform wear.) These statistics will be output to a CSV or console. We expect the new algorithm to show significantly smaller range and variance, and higher Jain's index, under all adversarial patterns compared to baseline.

- **Wear Distribution:** We will produce histograms or CDFs of block erase counts for each run. This visualizes how tightly clustered the wear is. For example, under the hot-spot pattern, baseline might show one block at 50 erases while many are near 0, whereas our algorithm might show all blocks around ~5 erases each. *Figures:* We plan to include bar graphs of per-block erases for small-scale runs, and distribution curves for larger runs to demonstrate the improvement.

- **Write Amplification:** The algorithm's overhead can be quantified by extra writes and erases incurred. We will count total *erase operations* performed and compare to baseline. Ideally, our algorithm might do slightly more erases (due to proactive leveling) but not excessively. We will also count how many *page migrations* or copies were done as part of wear leveling. This helps evaluate if the strategy is efficient. (We anticipate a modest increase in total writes/erases in exchange for huge gains in wear balance.) A table will present baseline vs WL algorithm WAF (Write Amplification Factor).

- **Performance (Throughput and Latency):** Using the fairness simulator and MQSim, we'll measure aggregate IOPS and latency. In fairness simulator, throughput is basically limited by the I/O arrival rate and channel bandwidth. We will ensure none of the workloads exceed the configured bandwidth such that results are comparable. Latency in fairness sim is simply service time (since there's no detailed queueing beyond what the scheduler imposes). We expect throughput to remain similar between baseline and WL algorithm, because our scheme tries to hide its overhead. However, if our algorithm occasionally triggers extra operations, it could *slightly* reduce effective throughput. We will verify if any drop is negligible. In MQSim, we'll look at per-tenant and overall throughput, plus latency distributions. Notably, we will examine **99th percentile latency**. In mixed workloads, baseline might suffer high tail latencies for the light tenant when the heavy tenant triggers GC, whereas our algorithm might reduce these events by smoothing out wear (and thus GC) across channels. We will gather latency traces from MQSim or use its stats to get tail latencies, and plot them.

- **Per-Flow Slowdown & Fairness:** For each tenant (user) in multi-tenant scenarios, compute the slowdown = (latency or turnaround time in shared run) / (latency if run alone on the device). Alternatively, use throughput ratio (throughput in shared / throughput alone). A perfectly fair system yields similar slowdowns for all tenants (ideally each >1 due to sharing, but equal). We will calculate each tenant's slowdown and then compute Jain's fairness index across those slowdown values. This indicates how fair the performance is. We anticipate our wear-leveling might incur a tiny penalty on the heavy writer (due to extra overhead or slightly throttling them during swaps) but protect the

light tenant, resulting in more equal slowdowns (thus higher fairness index) than baseline. Baseline might let the heavy writer dominate and the light one suffer if GC blocks it, or vice versa if the scheduler is strict. We'll show these metrics in a table or spider-chart.

- **Sensitivity Analysis:** We will also experiment with different algorithm settings (like the threshold Δ and check interval, the randomness level in block selection, etc.) to see the trade-offs. For example, one run could use a very aggressive leveling (small Δ) and show extremely uniform wear but maybe higher overhead, while another uses a larger Δ to allow some imbalance (like FlashBlox's approach [6] for performance) and show slightly lower overhead. This will inform the best configuration.

Each experiment will be run multiple times or with different random seeds to ensure consistency (especially for the Zipf and randomized selection patterns). The results will be compiled into CSV files for each metric. We will include those as tables and generate plots:

- *Table 4:* Wear leveling metrics for each workload (Baseline vs WL algorithm): listing min, max, variance, fairness index, etc.
- *Table 5:* Performance results for each workload (throughput, 99th latency, per-flow slowdowns, fairness index of slowdown).
- *Figure 4:* Erase count distribution (CDF) comparison for the Zipf workload with and without WL.
- *Figure 5:* Time-series of wear gap (max-min) during the run for FTL probing pattern – showing baseline rapidly increasing gap vs WL keeping it low (illustrating when swaps occur as the curve resets).
- *Figure 6:* 99th-percentile latency of a light tenant's requests over time in the mixed workload, comparing baseline vs WL (expected spikes in baseline when GC happens, vs a smoother line with our WL).

(Figures and tables are described here as placeholders; actual data will populate them in the final report.)

Trace Generation & Benchmarking Tools

To facilitate these experiments:
- We will write **custom trace generators** (in Python or using the existing `trace_gen.py` as a basis) for the specific patterns. For example, for the Zipf workload we can use Python's `numpy.random.zipf` to generate addresses. For mixed workloads, we might generate two traces and merge them by timestamp (ensuring the desired interleaving). The repository's `scripts/generate_traces.py` could be extended to produce these patterns systematically (perhaps by providing a mode or parameters for distribution shape, hot address count, etc.). We will version-control these synthetic trace scripts so experiments are reproducible.
- For running the simulations, we'll use the **automation helpers** in the repo (`run_matrix.py` or similar) to sweep configurations. For instance, run each trace with scheduler=RR (round-robin) vs scheduler=WFQ to see if scheduling interacts with wear leveling (shouldn't much, but worth checking if, say, SGFS which rotates user IDs might complement wear leveling by spreading load across channels logically).
- After simulation, we'll use the provided `plot_results.py` or custom matplotlib scripts to plot the results from CSV outputs. For MQSim, we may need to parse its output logs or extend it to output a CSV of relevant stats per run.
- We also plan to benchmark **performance overhead**: measure CPU time the simulator spends with our changes vs baseline, to ensure our added logic (which might sort blocks or iterate over arrays frequently)

doesn't slow the simulation excessively for large traces. If needed, we'll optimize the data structures (e.g., maintain a min-heap of wear counts for O(log N) selection of least-worn block).

Finally, we will combine all these results to demonstrate that our wear-leveling algorithm effectively counters the adversarial patterns: no single pattern can cause pathological wear imbalance or unfair slowdown. We expect to achieve near-perfect wear balance in all cases – akin to prior work that achieved **almost uniform wear with low overhead** [5] – thereby significantly extending device lifespan without sacrificing tenant QoS.

(Table 6 will compile a "lifetime improvement" estimate: e.g., how many extra cycles until first block failure in each scenario, baseline vs our WL – placeholder.)

Codex Prompt for Implementation

To assist a developer in coding this algorithm in the SSD fairness simulator, we can provide a clear prompt for GitHub Copilot or OpenAI Codex. The prompt should outline the modifications needed in `ssd.hpp` and `ssd.cpp` and provide guidance on implementing the core logic. For example:

```
**Task:** Implement a wear-leveling FTL extension in the SSD Fairness Simulator (C++17 codebase). We need to modify the SSD model to track flash block usage and evenly distribute writes across blocks (wear leveling), with an optional simple garbage collection. Below are the requirements and hints:

- The simulator currently does not model physical addresses or wear. We will introduce:
  - A constant for total flash blocks (e.g., 100 or make it configurable).
  - A constant for pages per block (e.g., 256).
  - Data structures: an array `wear_count` for block erase counts, initialize all to 0; an array to track how many pages are used in each block; perhaps a mapping from logical page to physical location (e.g., `std::unordered_map<long, std::pair<int,int>>` for LBA -> (block, page)).
- **Write dispatch logic:** In `SSD::dispatch` (or a new function it calls) handle write requests specially:
  1. Parse the `Request r.address` to derive a logical page number (e.g., use `r.address / 4096` if address is byte offset).
  2. If this LBA was written before (check the map), mark its old physical page as invalid (you may track invalid counts per block).
  3. Select a target block for the new write:
     - Find the block with the lowest `wear_count` that has free pages remaining. (If multiple, choose one with lowest wear; you can also pick randomly among them to avoid tie-break bias.)
     - Mark the next free page in that block as used for this write.
     - Update the LBA mapping to point to this new physical location.
  4. If the block becomes full (used pages == pages_per_block), handle it:
     - If GC is *not* enabled, immediately erase the block: increment `wear_count[block]++`, reset its used pages to 0, and mark all pages free
```

```
(essentially treating it as recycled). *Note:* This means data on that block is
lost unless moved; for simplicity, assume data is either invalid or saved
elsewhere. This gives an approximate wear cycle without full data movement.
     - If GC is enabled (optional), you would instead add the block to a list of
full blocks and later pick a victim block to erase when needed. (Implementing
actual GC with valid page copying is complex; you can simulate by choosing a
victim and just erasing it, assuming data was migrated.)
  5. Periodically (e.g., every 1000 writes or at simulation end), do a wear-
leveling check:
     - Find the block with max wear and min wear.
     - If (max - min) > threshold (say 5), then perform a swap:
       * Increment wear_count of the min-wear block (simulate erasing it one
extra time to level up).
       * (Optionally log that a wear-leveling swap happened between those
blocks.)
       * In a real implementation, we would copy data from min-wear to max-wear
block, etc., but here we can skip actual data movement since it's abstract. We
just need to adjust counts to reflect that the previously cold block was erased.
     - Repeat this check periodically.
- **Read requests:** can remain unchanged (no wear impact).
- **Timing:** Ensure the added logic doesn't break the timing model. The
`SSD::dispatch` returns a completion time; currently it's based on bandwidth. We
might not need to alter timing significantly, except maybe adding a tiny delay
for an erase or so. To keep it simple, we won't modify timing for wear leveling
in this simulator (assuming negligible impact).
- **Metrics:** Add new output fields (in `metrics.cpp` or related) to report at
least the final min, max, avg erase counts, and maybe the standard deviation.
This will help verify the WL effectiveness.

Now, proceed to implement the above in the code. Modify `include/ssd.hpp` to add
new members (wear_count array, etc.), and update `src/ssd.cpp` accordingly
(particularly the `SSD::dispatch` method).
```

The above prompt can be given to a coding AI along with the relevant portions of the codebase (for context) to generate an initial implementation. After getting the generated code, a developer should **review and test** it on simple scenarios (e.g., a trace of a single address being written repeatedly) to confirm that wear_count increases evenly and the simulator outputs make sense. This iterative approach (prompting, reviewing, testing) will yield a correct integration of our wear-leveling strategy into the fairness simulator's codebase.

Conclusion and Next Steps

We have proposed a comprehensive wear-leveling algorithm that proactively combats adversarial write patterns and ensures fairness in multi-tenant SSD use. We detailed how to implement it in both a simplified simulator and a full-featured one, and provided an experimental plan to validate its effectiveness. The next steps include coding the solution, running the described experiments, and iterating on the algorithm parameters based on the observed results. Ultimately, this work will demonstrate that with intelligent wear

leveling, an SSD can achieve **near-ideal lifetime with minimal performance disruption** even under worst-case workloads [5], thereby providing both reliability and fairness in shared storage environments.

---

[1] [3] [4] C:/Users/Muthu/Desktop/DB/DB.dvi
https://www-users.cse.umn.edu/classes/Spring-2020/csci5980/papers/SSD/Rejuvenator.pdf

[2] Device-Level Optimization Techniques for Solid-State Drives: A Survey
https://arxiv.org/html/2507.10573v1

[5] [6] microsoft.com
https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/fast17-final100.pdf

[7] [8] README.md
https://github.com/qchen4/ssd-fairness/blob/59caa4f80c25d9092767fd09d2d2a1c9210edf64/README.md